LAB 05 — D3.js: Interactive COVID-19 Scatterplot

Data Science and Data Visualization · International University
Pairs · 150 min in class (including introduction) · D3 v7

> **Your task:** Create an interactive geographic-coordinate **scatterplot** of historical confirmed COVID-19 cases. Each valid country/region record is one translucent circle. Users should discover its details by hovering — **not** by reading permanent labels.

Learning goals: (1) connect geographic coordinates to SVG positions; (2) attach and test D3 pointer events and tooltips; (3) evaluate visual correctness and change one requirement.

1. What you are building

Data	Visual representation
`Long` (longitude)	X-position, labeled axis and ticks
`Lat` (latitude)	Y-position, labeled axis and ticks
Confirmed cases	Circle **area**, through a square-root radius scale; use opacity for overlap
Country/region and case details	**Tooltip on hover**, no permanent labels

Date: 5 April 2020 (`05/04/2020` in DD/MM/YYYY; **`4/5/20`** in source CSV).
Source CSV: JHU CSSE confirmed global time series.

Scope: A longitude–latitude **scatterplot with axes**, as in the original lab; it is **not** a geographically projected world map. Note that longitude/latitude spacing is not equal physical distance everywhere.

2. Start here — before drawing anything

1. Complete `01_PreLab_Lab05_TaskFirst.md` (15–20 min) and run `examples/01_worked_interaction.html`.
2. Create `index.html`, `script.js`, `style.css`, `NOTES.md`. Run `python -m http.server 8000` inside your folder; open `http://localhost:8000`.
3. Inspect the real CSV **before using its fields**. Paste this snippet into `script.js` after loading D3 v7 in `index.html`:

```
const URL = 'https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/' +
  'csse_covid_19_data/csse_covid_19_time_series/' +
  'time_series_covid19_confirmed_global.csv';
d3.csv(URL).then(rows => {
  console.log('Source fields:', rows.columns);
  console.table(rows.slice(0, 3));
  console.log('Has target date?', rows.columns.includes('4/5/20'));
}).catch(error => console.error('CSV load failed:', error));
```

The expected geographical columns are `Province/State`, `Country/Region`, `Lat`, `Long`. Confirm them on the actual file. Numeric CSV values arrive as strings. **Do not assume a row always has a valid coordinate.** If the remote URL cannot be loaded, download the **same source CSV** into your project and use `d3.csv('./time_series_covid19_confirmed_global.csv')`.

THINK (5 min, no AI): Which records could have missing coordinates? Why might nearby circles overlap? What should a tooltip show? Agree on your answers before implementation. AI may help in BUILD/VERIFY/ADAPT, but you must test any suggestion.

3. Build — in this order

A. Prepare the data

- Extract values for `Country/Region`, `Province/State`, `Lat`, `Long`, and **`4/5/20`**.
- Check for blank fields **before** converting (JavaScript `Number('')` becomes 0). Convert latitude, longitude, and cases to numbers. Reject non-finite values, negative case counts, and out-of-range coordinates (longitude outside **−180..180**, latitude outside **−90..90**). Document how you handle missing case values or zeros.
- Keep **one record per valid source row** (country/region is not necessarily one row per country). Report loaded, valid and excluded counts in `NOTES.md`.

Check: Can you print three cleaned records with numeric coordinates and case counts?

B. Draw the scatterplot first

- Create an SVG with plot margins; draw **longitude** (−180..180) on X, **latitude** (−90..90) on Y, including ticks and clear axis labels.

- Use one `<circle>` per valid record; scale positions with `scaleLinear()` and use a `scaleSqrt()` for radius so circle **area**, not radius, conveys case count.
- Choose a readable radius range and partial opacity. Show the selected date, a title, units and data source. **Do not show permanent place labels.**

Check: Does positive latitude plot above the equator? Are all circles inside the chart area?

C. Make the chart interactive

- Attach `pointerenter`, `pointermove`, `pointerleave` using **D3 v7 `(event, d)`** callbacks.
- On hover, show a tooltip with country, province/state if available, latitude, longitude and **confirmed cases on 5 April 2020**; highlight the circle.
- Move the tooltip with the pointer and hide it when leaving. Make hover work on overlapping points as well as practical; do not add visible point labels.

Check: Hover three different circles. Does each tooltip report that circle's data? Does it disappear on leave?

4. Verify and adapt

Record **expected** → **actual** for these tests in `NOTES.md`:

Test	Expected result
V1 — data count	Rendered circles = valid data records
V2 — geography	Longitude 0 is central; northern latitude is above southern latitude
V3 — zero / invalid	Zero values and missing/invalid coordinates follow your documented rule; nothing becomes `NaN`
V4 — hover	Tooltip matches the hovered record; no permanent labels; hides on exit

Find and fix one bug: e.g., reversed Y, strings treated as numbers, wrong date column, stuck tooltip, or a circle count mismatch. Show the **failed observation** → **cause** → **change** → **retest**.

Required ADAPT: On click, **pin/unpin a tooltip** for a selected point (click again or press Escape to dismiss). Hovering other points must not silently overwrite the pinned content. Explain how you separated temporary hover state from selected state.

Optional stretch (original Lab 5): Add `d3.brush()` to select/highlight circles inside a rectangular brush. Report the selected record count; ensure the brush is usable alongside your tooltip.

5. Submit

Submit **one ZIP per pair** containing `index.html`, `script.js`, `style.css`, `NOTES.md` and any locally stored CSV file used. In `NOTES.md`, include: names/IDs; CSV/date/schema and validation counts; three THINK answers; V1–V4 results; one bug/fix; your click-to-pin design; and a brief AI-use disclosure (if applicable). Be ready to explain and modify an event handler during the Group Check.

Original course page requirement: include your name, student ID, course and lab titles, and the statement: “This is all my own work. I did not copy the code from any other source.” Make any required disclosure of AI or other assistance truthful.

Reading: Scott Murray, Interactive Data Visualization for the Web, Chapter 10 (Interactivity). The reading contains examples using older D3 event APIs.